

DriftGuard — Models & Methods: Full Mathematical Reference

Every model and method used in the pipeline, what it computes, and why it's here. Notation matches the TA-slide panel ("Metric / Drift detection / Adaptation + selector") so the two documents read consistently.

Part A — Detection models

A.1 Linear Discriminant Analysis (LDA)

What it does: Finds the linear combination of features that best separates classes, assuming each class is Gaussian with a *shared* covariance matrix.

The math: For two classes (benign vs. malicious), LDA models each class as a Gaussian $\mathcal{N}(\mu_k, \Sigma)$ with a common covariance Σ . Bayes' rule gives the discriminant function:

$$\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k$$

where π_k is the class prior. The predicted class is $\arg \max_k \delta_k(x)$. Because Σ is shared, the decision boundary between two classes is exactly **linear**:

$$\delta_1(x) - \delta_0(x) = w^\top x + b, \quad w = \Sigma^{-1}(\mu_1 - \mu_0)$$

Equivalently, LDA can be derived (Fisher's criterion) by finding the projection w that **maximises between-class variance relative to within-class variance**:

$$w^* = \arg \max_w \frac{w^\top S_B w}{w^\top S_W w}$$

where S_B is the between-class scatter and S_W the within-class scatter.

Why it's here: It's the cheapest possible baseline, and Cantone et al. (2024) found it was the model that generalised *best* cross-dataset — the linear decision boundary is too simple to overfit lab-specific quirks the way tree ensembles can.

A.2 Decision Tree

What it does: Recursively splits the feature space into regions that are increasingly pure with respect to the class label.

The math: At each node, the tree picks the feature j and threshold t that minimise impurity after the split. Two common impurity measures:

$$\text{Gini}(S) = 1 - \sum_k p_k^2 \quad \text{Entropy}(S) = - \sum_k p_k \log_2 p_k$$

where p_k is the proportion of class k in node S . The split is chosen to maximise the **information gain**:

$$\text{Gain}(S, j, t) = \text{Impurity}(S) - \frac{|S_L|}{|S|} \text{Impurity}(S_L) - \frac{|S_R|}{|S|} \text{Impurity}(S_R)$$

Splitting continues until a stopping criterion (max depth, min samples per leaf) is reached.

Why it's here: Simple, interpretable, and the direct building block of Random Forest — useful for isolating what ensembling actually adds.

A.3 Random Forest

What it does: Trains many decision trees on bootstrapped samples of the data, each using a random subset of

features at every split, then averages their predictions.

The math: Given B trees $T_1, \dots, T_B$, each trained on a bootstrap sample $\mathcal{D}_b$ (sampled with replacement from $\mathcal{D}$) with a random feature subset of size $m \approx \sqrt{p}$ considered at each split, the forest's prediction is:

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (\text{regression / probability}), \quad \hat{y}_{\text{RF}}(x) = \text{majority vote}\{T_b(x)\}_{b=1}^B \quad (\text{classification})$$

The randomness in both sampling (bagging) and feature selection decorrelates the trees, so averaging reduces variance without increasing bias:

$$\text{Var}(\hat{f}_{\text{RF}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

where σ^2 is a single tree's variance and ρ is the pairwise correlation between trees — lower ρ (from more randomness) drives the variance down further as B grows.

Why it's here: A strong, fast, well-understood tabular baseline and the most widely used model in the NIDS literature — the natural point of comparison against prior published work.

A.4 XGBoost

What it does: Builds trees **sequentially**, where each new tree corrects the errors of the ensemble so far, using gradient boosting with a regularised objective.

The math: The model is additive: $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$. XGBoost minimises a regularised loss using a **second-order Taylor expansion** around the current prediction:

$$\mathcal{L}^{(t)} \approx \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

where $g_i = \partial_{\hat{y}} \ell(y_i, \hat{y}_i^{(t-1)})$, $h_i = \partial_{\hat{y}}^2 \ell(y_i, \hat{y}_i^{(t-1)})$ are the first and second derivatives of the loss, and the regularisation term penalises tree complexity:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

(T = number of leaves, w_j = leaf weights). For a fixed tree structure, the optimal leaf weight and the resulting **gain** used to decide splits are:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \text{Gain} = \frac{1}{2} \left[\frac{(\sum_{I_L} g_i)^2}{\sum_{I_L} h_i + \lambda} + \frac{(\sum_{I_R} g_i)^2}{\sum_{I_R} h_i + \lambda} - \frac{(\sum_I g_i)^2}{\sum_I h_i + \lambda} \right] - \gamma$$

Why it's here: Usually the ceiling for tree-based methods on tabular data — the "best classical baseline" a reviewer will expect to see matched or beaten.

A.5 MLP (Multi-Layer Perceptron)

What it does: A basic feedforward neural network — needed because it's differentiable, which trees are not, making it compatible with the gradient-based adaptation methods (CORAL, DANN, AdaBN, Tent).

The math: Each layer applies an affine transform followed by a nonlinearity:

$$h^{(l)} = \sigma \left(W^{(l)} h^{(l-1)} + b^{(l)} \right), \quad h^{(0)} = x$$

with σ typically ReLU ($\sigma(z) = \max(0, z)$) for hidden layers and softmax for the output layer:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Trained by minimising cross-entropy loss via gradient descent:

$$\mathcal{L} = - \sum_k y_k \log \hat{y}_k, \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Why it's here: The simplest model that satisfies the "must be differentiable" requirement for later adaptation.

A.6 Autoencoder

What it does: Learns to compress and reconstruct **benign** traffic; attacks are flagged as flows the model reconstructs poorly.

The math: An encoder f_{θ} maps input to a low-dimensional code $z = f_{\theta}(x)$, and a decoder g_{ϕ} reconstructs $\hat{x} = g_{\phi}(z)$. Trained to minimise reconstruction error on benign data only:

$$\mathcal{L}_{\text{recon}} = \|x - g_{\phi}(f_{\theta}(x))\|_2^2$$

At inference, the **anomaly score** is the reconstruction error itself; a flow is flagged malicious if:

$$\|x - g_{\phi}(f_{\theta}(x))\|_2^2 > \tau$$

for some threshold τ (set on validation data, same spirit as the FPR@95%DR threshold below).

Why it's here: Represents a different detection paradigm (unsupervised/semi-supervised) than the other five supervised classifiers — useful when labelled attack examples are scarce or unreliable.

A.7 TabNet

What it does: A deep tabular model that uses sequential **attention** to pick which features matter at each decision step, rather than using all features at once like an MLP.

The math: TabNet processes input across N_{steps} decision steps. At step i , an **attentive transformer** produces a sparse feature mask using **sparsemax** (a sparse alternative to softmax that can output exact zeros):

$$M_i = \text{sparsemax}(P_{i-1} \cdot h_i(a_{i-1}))$$

where P_{i-1} is a prior-scale term that discourages re-selecting the same features every step:

$$P_i = \prod_{j \leq i} (\gamma - M_j)$$

The masked features $M_i \odot x$ are passed through a **feature transformer** to produce the step's contribution to the final decision, and outputs from all steps are aggregated (typically ReLU-summed) into the final prediction.

Why it's here: Chosen specifically because, unlike trees, it's differentiable (so Tent can update it at test time), and its step-wise architecture makes it a natural fit for the Reptile meta-learning setup — both need a model that can take a few gradient steps and adapt cleanly.

Part B — Evaluation metrics

(This is the exact formalism from the "1 · Metric" panel.)

Detection Rate (Recall) and False Positive Rate:

$$\text{DR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

FPR@95%DR — the metric actually reported. The threshold t^* is chosen **on source validation data only**:

$$t^* = \max\{t : \text{DR}_{\text{src}}(t) \geq 0.95\}$$

then **frozen** and applied, unchanged, to the target test set:

$$\text{FPR@95\%DR} = \text{FPR}_{\text{tgt}}(t^*)$$

This is why the diagram marks t^* as "fixed on source validation, frozen — never re-tuned on the target": re-tuning the threshold on target data would leak target labels into the evaluation.

MCC (Matthews Correlation Coefficient) — the secondary metric, robust to class imbalance:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Protocol: PR-AUC reported alongside as a threshold-free complement; 3 seeds, mean $\pm$ std; test data used exactly once.

Part C — Drift detection (no labels needed)

(From the "2 · Drift detection" panel — all three run on live, unlabelled traffic.)

C.1 Kolmogorov-Smirnov (KS) test, per feature

For feature j , compare its empirical CDF on the source data $F_{\text{src},j}$ against the live stream $F_{\text{live},j}$:

$$D_j = \max_x |F_{\text{src},j}(x) - F_{\text{live},j}(x)|$$

Feature j is flagged as drifted if its p-value $p_j < \alpha$, with α **Bonferroni-corrected** across all features tested simultaneously (dividing the significance threshold by the number of features, to control the false-alarm rate when running many tests at once).

C.2 Maximum Mean Discrepancy (MMD)

A **multivariate** test — catches shifts across combinations of features that a per-feature KS test would miss. Using kernel mean embeddings:

$$\text{MMD}^2 = \mathbb{E}[k(x, x')] + \mathbb{E}[k(y, y')] - 2 \mathbb{E}[k(x, y)]$$

with a typical choice being the RBF kernel:

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

A large MMD^2 means the source distribution (x 's) and the live distribution (y 's) are statistically distinguishable.

C.3 ADWIN (Adaptive Windowing), on model confidence

Monitors a stream (e.g. prediction confidence) and maintains a window W that is split into two sub-windows W_0, W_1 whenever their means differ by more than a data-driven bound:

$$|\mu_{W_0} - \mu_{W_1}| \geq \varepsilon$$

where ε comes from a Hoeffding-style bound:

$$\varepsilon = \sqrt{\frac{1}{2m} \ln \frac{4|W|}{\delta}}, \quad m = \left(\frac{1}{n_0} + \frac{1}{n_1}\right)^{-1}$$

(n_0, n_1 = sizes of the two sub-windows, δ = confidence parameter). When the bound is exceeded, ADWIN drops the older sub-window W_0 , effectively "forgetting" stale data and flagging a change point.

Why these three together: KS tells you *which specific feature* shifted (interpretable, but blind to correlations); MMD catches *joint* shifts across features that KS would miss; ADWIN turns either signal into a live, streaming change-point alarm rather than a one-off static test.

Part D — Adaptation methods

(From the "3 · Adaptation + selector" panel.)

D.1 Deep CORAL

Aligns the **second-order statistics** (covariance) of source and target feature representations:

$$\mathcal{L}_{\text{CORAL}} = \frac{\|C_S - C_T\|_F^2}{4d^2}$$

where C_S, C_T are the $d \times d$ feature covariance matrices of source and target batches, and $\|\cdot\|_F$ is the Frobenius norm. Minimising this loss (alongside the normal classification loss) pulls the target features toward the same "shape" as the source features, without needing target labels.

D.2 DANN (Domain-Adversarial Neural Network)

A minimax game between the label predictor and a domain classifier:

$$\min_{f,y} \max_d \mathcal{L}_y(f, y) - \lambda \mathcal{L}_d(f, d)$$

Implemented with a **gradient reversal layer**: during the forward pass it's the identity function, but during backpropagation it multiplies the gradient by $-\lambda$. This makes the feature extractor f get *worse* at letting the domain classifier d tell source from target apart, which pushes it toward domain-invariant features — while the label predictor y still learns to classify correctly.

D.3 AdaBN (Adaptive Batch Normalisation)

Simply **re-estimates** the batch-normalisation statistics using target-domain data, instead of retraining any weights:

$$\hat{x} = \frac{x - \mu_T}{\sqrt{\sigma_T^2 + \epsilon}}$$

where μ_T, σ_T^2 are computed fresh on target-domain batches (replacing the source-domain statistics the model was trained with). Cheap, needs no labels, no gradient updates to the main weights at all.

D.4 Tent (Test-time entropy minimisation)

Updates **only** the affine batch-norm parameters β, γ (scale and shift) at test time, by minimising the **entropy** of the model's own predictions on the live, unlabelled stream:

$$\min_{\beta, \gamma} H(\hat{y}) = - \sum_c \hat{y}_c \log \hat{y}_c$$

The intuition: a well-adapted model should be *confident* (low entropy) even without knowing the true label — so nudging the model toward confident predictions on live data is a usable, self-supervised training signal.

D.5 Few-shot labelling

No new formula — an analyst labels a small, detector-flagged sample of target flows (1%, 5%, or 10% of the target set), and the model is fine-tuned normally (standard supervised loss) on that small labelled set. Included as the ceiling on how much labelling cost buys you, for comparison against the label-free methods above.

D.6 Reptile (meta-learning)

Trains the base model **in advance**, across many source→target pairs, so that a handful of gradient steps adapts it well to *any new target*:

$$\theta \leftarrow \theta + \varepsilon(\phi_k - \theta)$$

where ϕ_k is the result of taking k ordinary gradient-descent steps starting from θ on a sampled task, and ε is the meta step size. Repeated over many sampled (source, target) tasks, this pulls θ toward a starting point that is *close*, in a few-gradient-steps sense, to good solutions for many different targets at once — unlike MAML, this needs no second-order gradients, since it only ever uses ordinary gradients within each task.

D.7 The selector (reward model — replaces RL)

Given the live drift state s (KS/MMD/ADWIN statistics, estimated FPR rise, labelling budget remaining), the selector picks the adapter a (from CORAL, DANN, AdaBN, Tent, few-shot, or the Reptile-initialised model) predicted to help most:

$$a^* = \arg \max_a f(s, a)$$

where $f(s, a)$ is a regressor trained on your own logged experiment outcomes — state and action in, measured $\Delta\text{FPR}@95\%\text{DR}$ out. This is a **supervised** prediction-and-greedy-selection step, not a reinforcement-learning agent: there's no exploration/exploitation trade-off, because the training data (every adapter's measured effect in every logged drift situation) already exists from your Phase B/D experiments.

This document is the mathematical companion to the DriftGuard Complete Project Guide — read alongside it, not as a replacement.